Massachvsetts Institute of Technology
Department of Electrical Engineering and Computer Science
6.863J/9.611J, Natural Language Processing

18 APR 18
William Mitchell, Scott Viteri, Mehmet Tugrul Savran
{williamm, sviteri, tugrul}@mit.edu

Fill In The Gap!

Mid-Course Check-In

1. Problem Statement

Filling in a gap within a sentence is a complex problem, and a prevalent domain of research in Natural Language Processing. It is an interesting challenge because we can often fill in a blank easily, but cannot describe exactly *how* we do it. An example of a *Fill In The Gaps* problem is realized below:

The sentence:
The President ___ the Vice President.
Choices:

A) Cat
B) Glanced
C) Met
D) Ate

Here, the language model should test each of the 4 possible choices, and pick the most appropriate option, choice C.

Ostensibly, the problem is to pick the most appropriate word from a corpus that will:

> **1)** yield a grammatical sentence -"The President cat the Vice President." is not a grammatical sentence, and
>
> **2)** yield a "natural" sentence -"The President ate the Vice President." is an unlikely sentence.

Accordingly, the computational pipeline breaks down into two major components: 1) coming up with a set of candidate words such as "Cat" or "Glanced" above. This is often called

the "corpus." 2) *iterating* through the corpus to pick the "best" fit. Now, let us describe what we mean by "best."

1.1 The "Best" Choice

We follow the notion that the best word is the one that minimizes surprise. Therefore, we will explore metrics such as entropy and information gain. Concretely, the language model will strive to pick the word that minimizes entropy. Naturally, the sentence "The President ate the Vice President." should yield a high cross-entropy with our English corpus, while "The President met the Vice President." should yield a lower cross-entropy.

In the next section we discuss several methodologies for the computational pipeline. Namely, we will test and confirm several methods we learned in 6.863 that will not work, such as N-Grams and the Penn Tree Bank. We hypothesize that N-Grams would not work because it cannot capture the *compositional structure;* Penn Tree Bank would not work because it is bound to fail on *feature agreement.* We end by proposing a composite method that **we think will work** that combines FCFG and wordnet.

Section 2 continues with such discussion in depth.

2. Methods

2.1 Corpus Selection and Sentence Generation

As Section 1 indicated, the first and the most straightforward part in the pipeline is the corpus. We utilized the Penn Tree Bank Corpus, which is easily accessible through NLTK. The key point with the corpus selection was to 1) have a corpus that is large and diverse and 2) be consistent with it (i.e. use the exact same corpus for the 3 different language models).

Next, using NLTK's sentence generators, we generated sentences from which we selected and removed a random word. The rest of the corpus is essentially *a list of options* to replace the removed word.

2.2 Language Model

The "brain" of the pipeline is the language model that fills in the gap in the sentence. We will train a vector representation of words using some corpus of sentences. We utilized 3 language models: the N-gram, the Skipgram, and the Penn Tree Bank CFG to find the minimum

entropy parse. We will compare and contrast several different methods that, according to what we learned in 6.863 so far, won't always work in this scenario: the N-Gram continuous bag-of-words, skip-gram and Penn Tree Bank CFG in which we will look for the lowest entropy parse.

For each vector embedding and each distance metric, we will show examples of situations that work, and situations that do not.

2.2.1 N-Grams

N-Grams are powerful tools in predicting the probability of a word w_n, the n^{th} word in the sentence, having observed a contiguous history h. That is, $P(w_n|h)$, where h is w_{n-1}, $w_{n-2,}$ …. Depending on the scope of the n-gram. For example, if n=2, the model looks at $P(w_n|w_{n-1})$.

Initially, we hypothesize that N-Grams will seldom fail because they will fail to capture the compositional structure. For example, in

He ate the lunch, __ words

Bag of words will not take into account the compositional structure. We need a language model that can predict a sentence of the form "He ate the lunch, *mumbling* words." We will test this hypothesis and explore the underlying phenomena if it fails.

2.2.2 Skipgrams

Skipgrams are pretty much N-Grams, but expanding on both sides. Hence instead of only considering the previously seen words (the Markov assumption in N-Grams), Skipgrams consider the following words that occur after the "blank" word.

The same hypothesis as in N-Gram holds for the Skipgram that it may fail with sentences having a compositional structure.

2.2.3 Penn Tree Bank CFG

Another interesting language model we took on to test was Penn Tree Bank's Context Free Grammar. Our methodology was as follows:

1) Define corpus as Penn Tree Bank's corpus (accessed by treebank.words())
2) Generate valid sentences from Penn Tree Bank
3) Take out (construct the "blank") a word randomly from each of these sentences

4) For each of these sentences, replace the blank with each of the words from the corpus
5) Compute the entropy for this "candidate" sentence
6) Return the minimum entropy candidate sentence

The entire methodology above is done by a Python script using NLTK tools, and also our lovely parser from 6.863 - we called the parser inside the script (at step 5).

An illustration is shown below:
An original sentence generated from PTB: Pierre Vinken , 61 years old , will join the board as a non-executive director Nov. 29 .
"Blanked" sentence: Pierre Vinken , 61 years old , will ___ the board as a nonexecutive director Nov. 29 .
Minimum entropy sentence (result): Pierre Vinken , 61 years old , will form the board as a non-executive director Nov. 29 .

2.2.4 FCFG

The clear option after using CFG's is to add features into the mix! We added tense and plurality features to power the Penn Tree Bank with an FCFG. For preliminary testing, we took a sample feature-CFG from the NLTK book, and picked filler words again using the entropy metric. The sample grammar was as follows:

```
% start S
# ###################
# Grammar Productions
# ###################
# S expansion productions
S -> NP[NUM=?n] VP[NUM=?n]
# NP expansion productions
NP[NUM=?n] -> N[NUM=?n]
NP[NUM=?n] -> PropN[NUM=?n]
NP[NUM=?n] -> Det[NUM=?n] N[NUM=?n]
NP[NUM=pl] -> N[NUM=pl]
# VP expansion productions
VP[TENSE=?t, NUM=?n] -> IV[TENSE=?t, NUM=?n]
VP[TENSE=?t, NUM=?n] -> TV[TENSE=?t, NUM=?n] NP
# ###################
```

```
# Lexical Productions
# ###################
Det[NUM=sg] -> 'this' | 'every'
Det[NUM=pl] -> 'these' | 'all'
Det -> 'the' | 'some' | 'several'
PropN[NUM=sg]-> 'Kim' | 'Jody'
N[NUM=sg] -> 'dog' | 'girl' | 'car' | 'child'
N[NUM=pl] -> 'dogs' | 'girls' | 'cars' | 'children'
IV[TENSE=pres,  NUM=sg] -> 'disappears' | 'walks'
TV[TENSE=pres, NUM=sg] -> 'sees' | 'likes'
IV[TENSE=pres,  NUM=pl] -> 'disappear' | 'walk'
TV[TENSE=pres, NUM=pl] -> 'see' | 'like'
IV[TENSE=past] -> 'disappeared' | 'walked'
TV[TENSE=past] -> 'saw' | 'liked'
```

First, we took this grammar and expanded it into CFG form with regex matches and a for loop. We picked the sentence ‘this dog walked’, and removed ‘walked’. We substituted in every possible terminal from the above fcfg, and checked the parse’s entropy using the provided parse file from lab 2. The sentence with the lowest entropy was ‘this dog disappears,’ which is semantically funky, but syntactically fine. We tried the same sentence using the PTB grammar, and got ‘this dog like.’ Both of these behaviors were as expected.

PTB CFG

```
6.863$ python3 max_ent_treebank.py
Found an improved sentence:  this dog this
Current entropy is:  15.817
Found an improved sentence:  this dog all
Current entropy is:  15.1721
Found an improved sentence:  this dog the
Current entropy is:  14.6765
Found an improved sentence:  this dog disappears
Current entropy is:  14.0972
Found an improved sentence:  this dog walk
Current entropy is:  14.0485
Found an improved sentence:  this dog see
Current entropy is:  13.8712
Found an improved sentence:  this dog like
Current entropy is:  13.8155
Minimum entropy found is:  13.8155
```

Minimum entropy sentence is: this dog like

NLTK FCFG

6.863$ python3 max_ent_treebank.py
Found an improved sentence: this dog disappears
Current entropy is: 2.86165
Minimum entropy found is: 2.86165
Minimum entropy sentence is: this dog disappears

We could continue making these proofs of inadequacy more rigorous, but maybe more promising direction is to augment the feature grammar with semantic information. Logical forms can be embedded into the NLP feature grammars, and this seems like a good way to encode a notion that 'he ate a hamburger' is better than 'he pickpocketed a hamburger'. We intend to try a few different methods of adding semantics into the picture. We hope that through working on lab 3, we will start gaining a better grasp of how to use semantics to help with this problem.

3. Goals/Extensions

We have proposed exploring four different methods for the fill-in-the-blank-problem: by using skip-gram, CFG's, and FCFG's augmented with semantic information. We have already tried a few preliminary methods to add semantics. We created a distance metric between sentences using wordnet, but this metric was useless because it only encoded pairwise relationships between individual words in each sentence -- so for our purposes would fill in 'tigers __ zebras' with 'tigers leopards zebras' rather than 'tigers eat zebras', for example. We even tried using combinatory categorial grammars to parse possible filled-in sentences, which we then mapped to semantic higher-order logic representations of those sentences. We picked the sentence fills whose parses generated the shortest lambda description -- a sort of minimum description length method. Preliminary results did not look too promising, but it is possible that we have not explored this path with enough rigor, and we plan testing this idea further.

A fortunate part of our task is that validation is fairly simple -- one can always take sentences, remove a word, and check if the method guessed correctly. But this may be overly strict. We could potentially take into account a few metrics -- did the method choose the correct word? Did the method choose the correct word in its top five choices? Did the method choose a fill-in word that was in the wordnet synonym set of the correct word? We can also do testing on the small set of words that can be generated by the NLTK book FCFG from above. I think this

has just enough structure to be both useful and easily interpretable. If we get to the point where we have a few separate feasible algorithms to fill in the blank, we can test on a corpus with greater structure as needed.

4. References

[1] S. Bird, E. Loper, and E. Klein. NLTK 3.2.5 Documentation. https://www.nltk.org/

[2] G. Miller, R. Beckwith, C. Fellbaum, D. Gross, and K. Miller. Introduction to WordNet: An On-Line Lexical Database. 1993. http://wordnetcode.princeton.edu/5papers.pdf

[3] ccg2lambda: Composing Semantic Representations Guided by CCG Derivations. https://github.com/mynlp/ccg2lambda

[4] The Penn Treebank: An Overview https://pdfs.semanticscholar.org/182c/4a4074e8577c7ba5cbbc52249e41270c8d64.pdf

[5] Feature Grammar Parsing. http://www.nltk.org/howto/featgram.html

[6] S. Bird, E. Loper, and E. Klein. Building Feature Based Grammars. 2014. http://www.nltk.org/book/ch09.html

[7] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, J. Dean. Distributed Representations of Words and Phrases and their Compositionality. 2013. https://papers.nips.cc/paper/5021-distributed-representations-of-words-and-phrases-and-their-compositionality.pdf

[8] M. Marcus, B. Santorini, M. Marcinkiewicz, and A. Taylor. Penn Treebank Sample. Philadelphia: Linguistic Data Consortium. 1999. https://catalog.ldc.upenn.edu/ldc99t42

[9] S. Bird, E. Loper, and E. Klein. Source Code for nltk.grammar. 2017. http://www.nltk.org/_modules/nltk/grammar.html